

# Module 3: Extracting and Visualizing Data

**Author:** Alexander Booth

**Date:** April 2026

---

## Overview

Module 3 focuses on getting data into your workflow — from web pages and financial APIs — and transforming it into DataFrames and visualizations.

You will use:

- **Web scraping** (HTML, Requests, BeautifulSoup)
- **Financial APIs** (yfinance)
- **Visualization tools** (Matplotlib / Plotly)

The core idea:

Data science starts with data.

This module gives you the tools to collect data from web pages and APIs, then visualize it so you can make decisions based on evidence.

---

## Why Data Extraction Matters

Real-world data rarely comes as a neat CSV file on your laptop.

It lives on:

- Web pages (tables, rankings, listings, revenue reports)
- Financial sources (stock prices, dividends, company metadata)
- APIs that provide structured JSON data

With web scraping and libraries like **yfinance**, you can:

- Automate what would take hours of manual copying
  - Build repeatable pipelines that refresh data
  - Combine multiple sources into a single analysis
  - Move from “data out there” to data in your DataFrame
- 

## What This Module Covers

### 1. HTML Fundamentals for Web Scraping

- Structure of a web page (DOCTYPE, html, head, body)
- Tag composition (opening/closing tags, attributes)
- Document tree (parent, child, sibling relationships)
- HTML tables (table, tr, th, td)

Understanding HTML structure is essential for reliable scraping.

---

### 2. Web Scraping with Requests and BeautifulSoup

- **Requests**: Download a web page (`requests.get(url)`)
- **BeautifulSoup**: Parse HTML into a navigable structure
- Key objects: Tag, NavigableString
- Navigating the tree (`.parent`, `.children`, `.next_sibling`)
- Filtering with `find()` and `find_all()`
- Reading attributes (`id`, `href`, `class`)

End-to-end workflow:

Download page → Parse HTML → Locate tags → Extract data → Build DataFrame

---

### 3. Web Scraping Practice Lab

Hands-on exercises include:

- Working with BeautifulSoup objects
  - Filtering by tag name and attributes
  - Extracting structured data from live/sample web pages
  - Converting scraped data into Pandas DataFrames
- 

### 4. Extracting Stock Data with yfinance

- Understanding stocks and ticker symbols
- Creating a Ticker object (e.g., `yf.Ticker("AAPL")`)
- Extracting:
  - Company metadata (`info`)
  - Historical share prices (`history()`)
  - Dividends

The result: structured financial data directly in Pandas.

---

### 5. Extracting Stock Data via Web Scraping

When no API is available:

- Send HTTP request with Requests

- Parse HTML with BeautifulSoup
- Locate revenue tables
- Extract rows and build a DataFrame

Key decision:

Use a library (yfinance) when available.

Use web scraping when data only exists on a web page.

---

## Final Assignment: Extracting and Visualizing Stock Data

You combine everything into a full pipeline.

You will:

1. Extract Tesla (TSLA) stock data with yfinance
2. Scrape Tesla revenue from a web page
3. Extract GameStop (GME) stock data with yfinance
4. Scrape GameStop revenue
5. Build a reusable graphing function
6. Plot stock price and revenue together

Pipeline:

Extract (API + scraping) → Combine → Visualize

This is your first full, real-world style data workflow.

---

## Key Takeaways

- HTML structure is critical for reliable web scraping.

- Requests fetches web content; BeautifulSoup parses and navigates it.
  - find() and find\_all() isolate the data you need.
  - yfinance provides fast access to stock data in Pandas.
  - Web scraping fills gaps when no API exists.
  - Data extraction is the foundation of data science workflows.
- 

## **The Data Scientist's Data-Collection Toolkit**

This module builds your toolkit.

The goal is not to memorize every BeautifulSoup method or yfinance parameter.

The goal is to:

- Identify where your data lives (API vs. web page)
- Fetch it using the right tool
- Convert it into DataFrames
- Visualize it
- Prepare it for analysis and modeling

Once you can reliably get data into your workflow, everything else becomes possible.